



مقدمة في الـ OOP

Object-Oriented Programming - البرمجة كائنية التوجه

طبيب يا جماعة، خرينا نبتدي من الأول. إيه هو الـ OOP أصلاً؟ هو مش مجرد طريقة كتابة كود، ده أسلوب حياة في البرمجة. الفكرة الأساسية إننا بنحول العالم الحقيقي لكود. بدل ما نكتب كود زي السيلفي (سطور ورا سطر)، بنقسم البرنامج لـ "كائنات" (Objects). كل كائن ليه صفات (Attributes) وسلوكيات (Methods).

ال OOP في الحياة العملية



فائدة النظام

النظام ده هو ال OOP. بيسهل علينا إدارة المشاريع الكبيرة جداً،
وخلانا نقدر نعيد استخدام الكود ونعدله بسهولة من غير ما نبوظ
حاجات تانية.



مثال: نظام إدارة مدرسة

تخيل إنك بتبرمج نظام لإدارة مدرسة. هتعمل كائن "طالب" و كائن
"مدرس" و كائن "فصل".
الطالب ليه اسم، وعمر، ورقم جلوس (**صفات**)، وبيقدر يذاكر، ويمتحن
(**سلوكيات**).

الجزء الأول: ال Encapsulation (التغليف)

ما هو التغليف؟

التغليف هو إننا نخبي تفاصيل تنفيذ الكود جوه الكائن، ونمنع أي حد بره الكائن إنه يعدل في بياناته بشكل مباشر.

في البرمجة: بنخلي البيانات (Attributes) وبنوفر Access (Methods (Getters & Setters) عشان نتحكم في الوصول للبيانات.

مثال المطعم

تخيل معايا إنك داخل مطعم. إنتا بتطلب أكل من الشيف (المطعم هو الكائن). إنتا متعرفش إزاي الشيف بيحضر الأكل جوه المطبخ (ده التفاصيل المخفية). وإنتا متقدرش تروح جوه المطبخ وتغير الوصفة بنفسك.

الطريقة الوحيدة إنك تطلب الأكل هي من خلال "Menu" (ال Methods العامة). لو الشيف غير طريقة الطهي، إنتا كزبون مش هتتأثر.

Encapsulation - مثال عملي (Java)

مثال بسيط علي كائن "حساب بنكي". له خاصية "الرصيد" private، محدث يقدر يغير الرصيد غير عن طريق الدوال بتاعت الإيداع والسحب:

```
public class BankAccount {  
    // البيانات مخفية (Private)  
    private String accountHolderName;  
    private double balance;  
  
    // Constructor عشان نعمل الحساب  
    public BankAccount(String name, double initialBalance) {  
        this.accountHolderName = name;  
        if (initialBalance > 0) {  
            this.balance = initialBalance;  
        } else {  
            this.balance = 0;  
        }  
    }  
  
    // بتاعنا Menu الـ Public Methods دول الـ  
    public void deposit(double amount) {  
        if (amount > 0) {  
            balance += amount;  
            System.out.println("تم إيداع: " + amount);  
        } else {  
            System.out.println("المبلغ لازم يكون أكبر من صفر");  
        }  
    }  
}
```


الجزء الثاني: ال Inheritance (الوراثة)

مفهوم الوراثة

الوراثة في البرمجة زيها زي الوراثة في الحياة، بناخد صفات وآباء من حد ثاني.

الفكرة إننا نقدر نعمل "كلاس أب" (Superclass) فيه الخصائص المشتركة، و "كلاس ابن" (Subclass) يرث كل حاجة من الأب ويضيف حاجات جديدة أو يعدل في الحاجات الموجودة.

مثال الموظفين والمدير

تخيل معايا إننا بنعمل برنامج للموظفين. كل الموظفين عندهم اسم، ومعرف، ومرتب أساسي.

بس في نوع ثاني من الموظفين اسمه "Manager" (مدير)، هو بردو موظف، لكن ليه حاجات زيادة زي "بدل إدارة" و "مكافأة". بدل ما نكرر كتابة الاسم والمعرف والمكتب في كلاس المدير، هنورثهم من كلاس الموظف الأساسي.

Inheritance - مثال عملي (Java)

```
// الأب Superclass ده ال
class Employee {
    String name;
    int id;
    double baseSalary;

    public Employee(String name, int id, double baseSalary) {
        this.name = name;
        this.id = id;
        this.baseSalary = baseSalary;
    }

    public double calculateSalary() { return baseSalary; }
    public void displayInfo() { System.out.println("الاسم: " + name + ", ID: " + id); }
}

// الابن Subclass بييمتد من Employee
class Manager extends Employee {
    double bonus;
    double allowance; // بدل إدارة

    public Manager(String name, int id, double baseSalary, double bonus, double allowance) {
```


الجزء الثالث: ال Polymorphism (تعدد الأشكال)

الكلمة دي مرعبة شوية، بس معناها بسيط: "القدرة على التعامل مع كائنات من أنواع مختلفة، كأنها من نوع واحد، مع الحفاظ على السلوك الخاص بكل نوع". أقدر أتعامل مع ال Manager كأنه Employee، لكن لما أطلب منه يحسب المرتب، يحسب مرتبه هو مش الموظف العادي.

```
public class Main {
    public static void main(String[] args) {
        Employee emp1 = new Employee("5000", 101, "أحمد");
        Manager mgr1 = new Manager("1500", 2000, 8000, 202, "سارة");

        // Employee التعامل مع الكل كـ
        Employee emp2 = new Manager("1000", 1500, 7000, 303, "خالد");

        // هنا بيظهر تعدد الأشكال
        System.out.println("مرتب أحمد: " + emp1.calculateSalary()); // 5000
        System.out.println("مرتب سارة: " + mgr1.calculateSalary()); // 11500 (8000+2000+1500)
        System.out.println("مرتب خالد (كـ Employee): " + emp2.calculateSalary()); // 9500 (7000+1500+1000) → Manager شغال بطريقة
    }
}
```


الجزء الرابع: ال Abstraction (التجريد)

التجريد هو إننا نركز على "إيه" الكائن يعمل، من غير ما نهتم بـ "إزاي" يعمل. بنستخدمه عشان نخفي تعقيد التنفيذ ونوفر واجهة بسيطة وسهلة للمستخدم.

Interfaces

واجهة بتحدد "العقد" أو "الاتفاق" اللي أي كلاس هيمضيه. بتقول للكلاس: "إننا لازم تنفذ الدوال الفلانية"، من غير ما تهتم إزاي هتنفذها. دي قمة التجريد.

Abstract Classes

كلاس مش بنقدر نعمل منه Object مباشر، بنستخدمه كـ "تصميم" أو "قالب" للكلاسات اللي هترث منه. ممكن يحتوي على Methods مجردة (من غير تنفيذ) و Methods عادية (منفذة).

مثال: نظام تشغيل السيارات لازم يعرف يعمل

startEngine()

stopEngine()

. لكن طريقة تشغيل محرك السيارة الكهربائية غير البنزين. النظام مش مهتم بالطريقة، المهم إن السيارة تفهم أمر التشغيل وتنفذه بطريقتها.

Abstraction - مثال عملي (Java)

```
// 1. ⚡ Abstract Class
abstract class Vehicle {
    String model;
    public Vehicle(String model) { this.model = model; }
    public void displayModel() { System.out.println("موديل العربية: " + model); }
    public abstract void startEngine(); // Method (مش منفذ) مجرد
}

// 2. ⚡ Interface
interface GPS {
    void navigateTo(String destination);
}

// ⚡ GPS Interface وبيطبق Vehicle كلاس ابن من
class ElectricCar extends Vehicle implements GPS {
    int batteryLevel;
    public ElectricCar(String model, int batteryLevel) {
        super(model);
        this.batteryLevel = batteryLevel;
    }
    @Override
    public void startEngine() {
```